

C#

GameManager.cs

Player.cs

Pipes.cs

Spawner.cs

SoundManager.cs

Parallax.cs

Flying Bird

ソースコード一覧

全 6 ファイルー 各スクリプトの役割と
実装コード

*Unity C# / MonoBehaviour / カジュアル2D
ゲーム*

GameManager / Player / Pipes / Spawner /
SoundManager / Parallax

GameManager.cs

Part 1/2

1 / 6

ゲーム全体の状態管理 – シングルトンパターンでゲーム内に1つだけ存在

```
using UnityEngine;
using UnityEngine.UI;

public class GameManager : MonoBehaviour
{
    public static GameManager Instance { get; private set; }

    [Header("UI関連")]
    public Text scoreText;          // スコア表示ラベル
    public Text titleText;          // タイトルテキスト
    public GameObject playButton;   // Playボタン
    public GameObject gameOver;     // ゲームオーバーパネル

    [Header("プレイヤー関連")]
    public Player player;

    private int score;

    private void Awake()
    {
        if (Instance == null)
        {
            Instance = this;
        }
        else
        {
            Destroy(gameObject);
            return;
        }
        Application.targetFrameRate = 60;
        Title(); // 起動時はタイトル画面
    }
}
```

ゲーム全体の状態管理ー シングルトンパターンでゲーム内に1つだけ存在

```
public void Play ()
{
    SoundManager.Instance.StopGameOverSE ();
    SoundManager.Instance.PlaySE (SoundManager.Instance.clickClip);
    titleText.gameObject.SetActive (false);
    playButton.SetActive (false);
    gameOver.SetActive (false);
    score = 0;
    scoreText.text = score.ToString ();
    Time.timeScale = 1f;
    player.StartGame ();
    Pipes[] pipes = FindObjectsOfType<Pipes> ();
    for (int i = 0; pipes != null && i < pipes.Length; i++)
        Destroy(pipes[i].gameObject); // 既存パイプを全削除
}

public void Pause() { Time.timeScale = 0f; player.enabled = false; }

public void Title()
{
    titleText.gameObject.SetActive (true);
    playButton.SetActive (true);
    gameOver.SetActive (false);
    player.enabled = false;
    Time.timeScale = 0f;
}

public void GameOver ()
{
    gameOver.SetActive (true);
    playButton.SetActive (true);
    SoundManager.Instance.PlayGameOverSE ();
    Pause ();
}

public void IncreaseScore ()
{
    score++;
    scoreText.text = score.ToString ();
}
}
```

Player.cs

2 / 6

鳥の移動・重力・傾きを制御ー タップで上昇、重力で自然に落下

```
// Player.cs -- Update()メソッドの抜粋 (フィールドはインスペクターで確認)
```

```
// 主要フィールド
```

```
public float strength = 5f;      // ジャンプ力
public float gravity  = -9.8f;   // 重力
public float tilt     = 5f;      // 傾きスケール
private Vector3 direction;      // 移動方向ベクトル
public AudioClip jumpClip;      // ジャンプSE
```

```
private void Update()
```

```
{
    // タップ / クリックで上昇
    if (Input.GetMouseButtonDown(0))
    {
        direction = Vector3.up * strength;
        SoundManager.Instance.PlaySE(jumpClip);
    }

    // 重力をフレーム毎に加算
    direction.y += gravity * Time.deltaTime;
    transform.position += direction * Time.deltaTime;

    // y方向の速度に応じて傾く
    Vector3 rot = transform.eulerAngles;
    rot.z = direction.y * tilt;
    transform.eulerAngles = rot;
}
```

パイプ（障害物）の移動・自動削除ー画面外へ出たら Destroy()

```
using UnityEngine;

public class Pipes : MonoBehaviour
{
    // スクロール速度（インスペクターから調整可能）
    public float speed = 5f;

    // 画面左端のx座標
    private float LeftEdge;

    private void Start()
    {
        // カメラの左端をワールド座標で取得
        LeftEdge = Camera.main.ScreenToWorldPoint(Vector3.zero).x - 1f;
    }

    private void Update()
    {
        // 毎フレーム左方向に移動
        transform.position += Vector3.left * speed * Time.deltaTime;

        // 画面外に出たら自己を削除
        if (transform.position.x < LeftEdge)
            Destroy(gameObject);
    }
}
```

Spawner.cs

4 / 6

パイプを定期的に生成 - InvokeRepeating で指定間隔ごとにSpawn() を呼び出す

```
// Spawner.cs -- 抜粋 (設定値 + OnEnable + Spawn)

public GameObject prefab;           // 生成するパイプ Prefab
public float spawnRate = 1.5f;     // 生成間隔 (秒)
public float minHeight = -1f;      // Y最小オフセット
public float maxHeight = 3.5f;     // Y最大オフセット

// ゲームプレイ開始時に呼び出される
private void OnEnable()
{
    // spawnRate秒待ってリピート実行
    InvokeRepeating(nameof(Spawn), spawnRate, spawnRate);
}

private void OnDisable()
{
    CancelInvoke(nameof(Spawn));
}

private void Spawn()
{
    // スポーナーの位置に Prefab をインスタンス化
    GameObject p = Instantiate(prefab,
                                transform.position,
                                Quaternion.identity);

    // Y座標をランダムにシフト
    p.transform.position += Vector3.up *
        Random.Range(minHeight, maxHeight);
}
```

SoundManager.cs

5 / 6

SE・ゲームオーバー音の再生・停止を一元管理ー シングルトンで全シーンからアクセス

```
// SoundManager.cs  --  抜粋 (フィールド + Play / Stopメソッド)

private AudioSource seSource;      // SE専用音源
public AudioClip gameOverClip;     // ゲームオーバー音
public AudioClip clickClip;        // クリック音

// 通常SE: PlayOneShot -> 重ね再生可能 / 停止不可
public void PlaySE(AudioClip clip)
{
    seSource.PlayOneShot(clip);
}

// ゲームオーバー音: clipを指定して Play -> 停止可能
public void PlayGameOverSE()
{
    seSource.clip = gameOverClip;
    seSource.loop = false;
    seSource.Play();
}

// 再スタート時に呼び出して GameOver音を切る
public void StopGameOverSE()
{
    if (seSource.isPlaying && seSource.clip == gameOverClip)
    {
        seSource.Stop();
    }
}
}
```

背景のループスクロールーUVオフセットを少しずつずらして無限ループを実現

```
using UnityEngine;

public class Parallax : MonoBehaviour
{
    // UVオフセットを操作するための MeshRenderer
    private MeshRenderer meshRenderer;

    // スクロール速度 (Inspectorで調整)
    public float animationSpeed = 1f;

    private void Awake()
    {
        meshRenderer = GetComponent<MeshRenderer>();
    }

    private void Update()
    {
        // 現在の UV オフセットを取得
        Vector2 offset = meshRenderer.material.mainTextureOffset;

        // 毎フレーム x 方向にアニメ速度分だけずらす
        offset += new Vector2(animationSpeed * Time.deltaTime, 0);

        // 更新したオフセットをマテリアルに反映
        meshRenderer.material.mainTextureOffset = offset;
    }
}
```